

Seja x um valor inteiro, e v um vetor de inteiros

Problema 4: escrever para encontrar a posição de x no vetor V , caso x esteja em V

1º

int tam; int pos

int vet[tam]

int x

for (int i = 0; i < tam; i++)

if (vet[i] == x)

pos = i

2º pesquisa binária

3º

int v[tam]

Sol. Simb:

① Aleatório $\rightarrow$ não garante que a resposta será a mais rápida

② Sequencial $\rightarrow$ Procura todos os elementos; tem garantia

③ Busca Binária $\rightarrow$ Se a árvore binária for degenerada para uma (Além de uma ABP em ordem)
 $\rightarrow$ deve ser feita a partir de um vetor que já está ordenado

④ Host | Estrutura de dados "avanzada"

Seja x um valor inteiro e L um vetor de inteiros

Problema 4: escrever (C/C++) para identificar a posição de x em L como inteiro. Faça usando

a) Busca Sequencial

se $x \in L$ ou $x \notin L$

int sequencial (int* L, int n, int x)

int i = 0; // 2 operações

while ((i < n) && (L[i] != x)) {

i++

} return i;

$4n + 1$

Ex Silvio $\rightarrow$ Verifique o número de operações

Como o elemento esteja presente
não precisa garantir o tamanho do vetor

$$x \notin L \rightarrow 2 + \sum_{i=0}^{n-1} 1 + 1 = n+4$$

$$x \in L \rightarrow 2 + \sum_{i=0}^{n-2} 2 + 2 + 1 = 2n+2$$

Ex: Como reduzir o número de operações caso $x \notin L$

- Garantir que o vetor tem uma posição a mais
porque x não está no vetor

se $x \in L$ ou $x \notin L$

int sequential (int* L, int n, int x)

int i = 0; // 2 operações

L[n] = x;

while (L[i] != x)

 i++;

if (i == n) return 1;

else return 1;

$$2 + 1 + \sum_{i=0}^{n-1} 2 + 1 + 2 = 2n+6$$

Ex:

Menor elemento

Maior elemento

Menor e maior do array

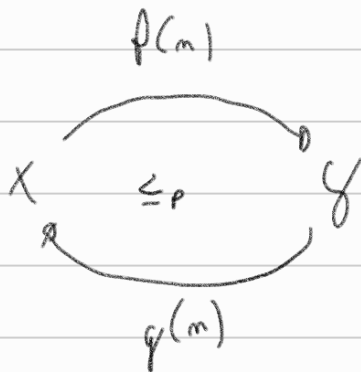
Solução que minimize o número de operações

```

int posMemor (int *v, int tam) {
    int p = 0;
    for (int j = 1; j < tam; j++)
        if (v[p] > v[j]) p = j;

    return p;
}

```



```

int maior (int *v, int tam) {
    int p = 0;
    for (int i = 0; i < tam; i++)
        v[i] = v[i] * (-1);
    p = posMemor(v, tam);
    return (-1) * v[p];
}

```

EX:

algoritmo e

custo computacional

maior e menor elemento

i	i+1									
4	3	1	6	9	5	0	2	7	9	
3	4	1	6	5	9	0	2	7	9	

Comparar os menores

Comparar os maiores

```
void minMem (int * v, int tam) {
```

```
    int m=0, M=1;
```

```
    for (int i=0; i < tam; i+=2)
```

```
        if (v[i] > v[i+1]) {
```

```
            swap(v[i], v[i+1]) // swap é necessário?
        }
```

```
        if (v[m] > v[i]) m=i;
```

```
        if (v[M] > v[i+1]) M=i+1;
```

```
}
```

$$\sum_{i=1}^n 2 = 2n$$

complexidade

$$f(n) = O(g(n)) \text{ se}$$

$$\exists c > 0, m_0 > 0 \mid f(n) \leq c \cdot g(n), \forall n \geq m_0$$

Ponto a partir de qual valor

é melhor usar um algoritmo

ou outro

$$f(n) = \frac{3n}{2}$$

$$\frac{3n}{2} \leq c \cdot n^2, \forall n \geq m_0$$

$$g(n) = n^2$$

$$f(n) = O(g(n))?$$

$$c \geq \frac{3}{2n} \begin{cases} m_0 = 1 \\ c = 2 \end{cases}$$

Esger com $c=0$ tendo a mesma constante

mesma para um infinito

Exº:

Seja $f_1(n) = O(g_1(n))$ e
 $f_2(n) = O(g_2(n))$,

mostre que

i) $f_1(n) + f_2(n) = O(g_1(n) + g_2(n))$
ii) $f_1(n) + f_2(n) = O(\max(g_1(n), g_2(n)))$

② Se $f_1(n) = O(g(n))$ e $f_2(n) = O(g(n))$
 $f_2(n) = O(g(n)) \rightarrow f_2(n) \in O(g(n))$

então $f_1(n) = f_2(n)$?

não é *o Computo que possui funções com variáveis similares*

mesclhamos

um igual

e tem um abuso de notação

Resp ①

ii

$$f_1(n) = O(g_1(n)) \Leftrightarrow \exists c_1, n_1, n_1 > 0 \mid f_1(n) \leq c_1 \cdot g_1(n), \forall n > n_1$$

$$f_2(n) = O(g_2(n)) \Leftrightarrow \exists c_2, n_2, n_2 > 0 \mid f_2(n) \leq c_2 \cdot g_2(n), \forall n > n_2$$

$$\underbrace{f_1(n)}_{1^\circ} + \underbrace{f_2(n)}_{2^\circ \text{ parte}} \leq c_1 \cdot g_1(n) + c_2 \cdot g_2(n) \leq c_1 \cdot g_1(n) + c_2 \cdot g_2(n)$$

então podemos afirmar que

$$f_1(n) + f_2(n) \leq c_1 \cdot g_1(n) + c_2 \cdot g_2(n)$$

podemos falar também que $\leq c_3 \cdot g_1(n) + c_3 \cdot g_2(n)$

sabendo que $c_3 = c_1 + c_2$

Se somarmos as

constantes a que acontece se somarmos os pontos de início

$$f_1(n) + f_2(n) \leq c_3 \cdot (g_1(n) + g_2(n)), \forall n_3 > n_1 + n_2$$

$$f_1(n) + f_2(n) = O(g_1(n) + g_2(n))$$

Resp ① ju

$$f_1(n) = O(g_1(n)) \Leftrightarrow \exists c_1, n_1 > 0 \mid f_1(n) \leq c_1 \cdot g_1(n),$$

$$f_2(n) = O(g_2(n)) \Leftrightarrow \exists c_2, n_2 > 0 \mid f_2(n) \leq c_2 \cdot g_2(n), \forall n > n_2$$

1º 2º parte

$$f_1(n) + f_2(n) \leq c_1 \cdot g_1(n) + f_2(n) \leq c_1 \cdot g_1(n) + c_2 \cdot g_2(n) \leq \boxed{c_1 \max(g_1(n), g_2(n)) + c_2 \max(g_1(n), g_2(n))}$$

então podemos afirmar que

$$f_1(n) + f_2(n) \leq c_1 \cdot g_1(n) + c_2 \cdot g_2(n) \quad \text{sabendo que } c_3 = c_1 + c_2$$

podemos falar também que $\leq \underbrace{c_1}_{1(n)} \cdot \underbrace{g_1(n)}_{2(n)} + \underbrace{c_2}_{2(n)} \cdot \underbrace{g_2(n)}_{K(n)}$

Km
Se renomear as constantes a que aparece se renomear as pontas de início

$$f_1(n) + f_2(n) \leq (c_1 + c_2) \cdot K(n), \forall n_3 > n_1 + n_2$$

$$f_1(n) + f_2(n) = O(\max(g_1(n), g_2(n)))$$

Se eu soma duas coisas
a ordem de complexidade pode ser maior delas
que é basicamente a maior taxa de crescimento existem
porém o tempo computacional deve ser considerado
a soma entre elas

USAR A DEFINIÇÃO

Toda função tem a ordem de complexidade exponencial

C

$$O(n) \leq O(n^2)$$

Verdadeiro ou falso? Mostre

$$2m^2 + m \leq 1$$

$$-1 \leq m \leq \frac{1}{2}$$

a) $2m^2 + m = O(1)$ $2m^2 + m = 1$

Para que o custo seja 1, m deve ser em -1 ou $0,5$

$$f_1(m) = O(g_1(m)) \Leftrightarrow \exists c > 0, m_0 > 0 \mid f_1(m) \leq c \cdot g_1(m), \forall m > m_0$$

$$2m^2 + m = O(1) \Leftrightarrow c > 0, m > 0 \quad \text{X}$$

$$\exists c > 0, m > 0 \mid 2m^2 + m \leq 1 \cdot c, \forall m > m_0$$

$$c > \frac{2m^2 + m}{1} \rightarrow \text{tende ao infinito logo } \nexists c$$

b) $2m^2 + m = O(m)$

c) $2m^2 + m = O(m^2)$

$$\exists c > 0, m > 0 \mid 2m^2 + m \leq m \cdot c, \forall m > m_0$$

$$c > \frac{2m^2 + m}{m}$$

1ª Encontrei por de constantes

Encontrei por de constantes, se

$$c > \frac{2m+1}{m}$$

$$m_0 = 1 \quad m_0 = 2 \quad m_0 = 1000$$

sem saber é verdadeiro

$$c: 3$$

$$c: 2,5 \quad c: 2,001$$

d) $2m^2 + m = O(m^3)$

$$c > \frac{2m^2 + m}{m^3}$$

$$c > \frac{2}{m} + \frac{1}{m^2} \rightarrow \frac{2m+1}{m^2} \quad c > \frac{2+1}{1} = 3$$

Logo achei por de constante não é verdadeiro

e) $2^m = O(3^m)$

se fosse log teria problema

$$c > \frac{2^m}{3^m} \rightarrow c > \left(\frac{2}{3}\right)^m$$

Como $\frac{2}{3} < 1$ tende a 0

como tende a 0 existe uma constante

$$m_0 = 0$$

$$c = 0,7$$

f) $3^m = O(2^m)$

$$\frac{3^m}{2^m} = \left(\frac{3}{2}\right)^m \quad \text{Como } \frac{3}{2} > 1, \text{ tende ao } \infty, \text{ logo não existe constante}$$

$$c > \left(\frac{3}{2}\right)^m$$

2.

seja $f(n) = O(g(n))$ e k uma constante. Mostre que $K \cdot f(n) = O(g(n))$
 (é possível?)

Relação de recorrência

$$T(n) = \begin{cases} T(n/2) + 1, & n \geq 1 \\ 1, & n = 0 \end{cases}$$

então $T(n) = O(\log(n))$?

$$C \gg \frac{n}{\log(n)}$$

$$T(n) = \begin{cases} T(n-1) + 1, & n \geq 1 \\ 1, & n = 0 \end{cases}$$

então $T(n) = O(n)$?

$$T(n) = \begin{cases} T(n-1) + n, & n \geq 1 \\ 1, & n = 0 \end{cases}$$

então $T(n) = O(n^2)$?

$$T(n) = \begin{cases} 2T(n/2) + 1, & n \geq 1 \\ 1, & n = 0 \end{cases}$$

então $T(n) = O(n)$?

$$T(m) = \begin{cases} T(m-1) + 1, & m > 1 \\ 1, & m = 1 \end{cases}$$

$$m-i \in [1, m]$$

$$m-i=1 \Rightarrow i=m-1$$

$$m-i=m \Rightarrow i=0$$

$$T(m-0) = T(m-1) + 1$$

$$T(m-1) = T(m-2) + 1$$

$$T(m-2) = T(m-3) + 1$$

$$\vdots$$

$$T(m-i) = T(m-i-1) + 1$$

$$\vdots$$

$$T(2) = T(1) + 1$$

$$T(1) = 1$$

$$T(m) = 1 + 1 + \dots + 1 = \sum_{i=0}^{m-1} 1 = m-1+1 = m$$

$$T(m) = \begin{cases} T(m-1) + m, & m > 1 \\ 1, & m = 1 \end{cases}$$

$$T(m-0) = T(m-1) + m-0$$

$$T(m-1) = T(m-2) + m-1$$

$$T(m) = \sum_{i=0}^{m-1} m-i$$

$$\sum_{i=x}^y c = c(y-x+1)$$

$$\sum_{i=x}^y i = \frac{(y+x)(y-x+1)}{2}$$

$$\sum_{i=0}^{m-1} m-i = \sum_{i=0}^{m-1} m - \sum_{i=0}^{m-1} i$$

$$\frac{m(m-1-0+1)}{2} - \frac{(m-1+0)\left(\frac{m}{2}\right)}{2} = \frac{m^2 - (m-1)m}{2} = \frac{2m^2 - (m-1)m}{2}$$

Formule
fechadas

$$\frac{2m^2 - m^2 + m}{2} = \frac{m^2 + m}{2}$$

Algoritmo $\rightarrow$ expressão

↓

formule fechadas

↓

ordem

Implemente a busca binária de forma recursiva em um vetor ordenado de n elementos

Encontre a ordem de complexidade

obs: m é potência de 2

$x = 6$

1 2 3 4 5 6 7 8 9 10

bool buscaBinaria() {

int $x = 5$

int $vet[8] = \{1, 2, 3, 4, 5, 6, 7, 8\}$

return buscaBinaria($x, 8/2, 7, 0$)

}

bool buscaBinaria(int x , int $meio$, int fim , int $inicio$) {

$q(m) = \begin{cases} T(m/2) + 2, & m > 1 \\ T(1), & m = 1 \end{cases}$

$q(1), m = 1$

$$T(n) = T(n/2) + C$$

$$T(n/2) = T(n/4) + C$$

$$T(n/2^i) = T(n/2^{i+1}) + C$$

$$T(2) = C$$

$$T(4) = 1$$

$$T(n) = \underbrace{C + C + C + \dots + C}_{\log_2 n} + 1$$

$$\frac{n}{2^i} \begin{cases} \nearrow n \\ \searrow 2 \end{cases}$$

$$\frac{n}{2^i} = n \cdot 2^{-i}$$

$$\frac{n}{2^i} = 2 \Rightarrow n = 2$$

$$\log_2 n = \log_2 2^{i+1}$$

$$i+1 = \log_2 n$$

$$i = \log_2 n - 1$$

Seja $G = (V, E)$ um grafo não direcionado. Mostre que cobertura de vértice e conjunto independente são equivalentes.

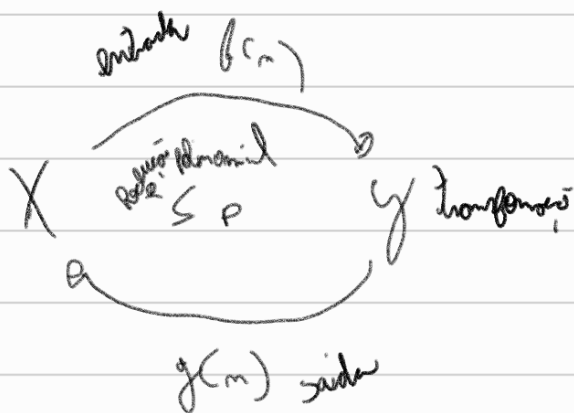
Conjunto indep. } não tem vértices adjacentes

Um subconjunto $S \subseteq V$ é independente se para todo par de vértice não existe uma aresta $(u, v) \in E$

Cobertura de vértice

é um conjunto de vértice tal que cada aresta no grafo é incidente a pelo menos um vértice do conjunto

Resolução:



Curto polinomial

$$X \leq_p Y \text{ e } Y \leq_p X$$



$X \equiv_p Y$ Logo são equivalentes

- Se não há solução polinomial para X e $X \leq_p Y$, então não há solução polinomial para Y
 $\rightarrow X$ está "espelhado" a dificuldade para Y

$$CI \equiv_p CV$$

$$\Rightarrow CI \leq_p CV \text{ e } CV \leq_p CI$$

Prova matematicamente

$$f(m) \equiv_p G(m)$$

Se S é CI então $V-S$ é CV e vice-versa
equivalência nos dois sentidos

Se CI então $V-S$ é CV

Seja S uma CI

$\Rightarrow e = (u, v) \in E \Rightarrow u \notin S \text{ ou } v \notin S$

então, $u \in V-S$ ou $v \in V-S$
portanto $V-S$ é CV

2º pte

Seja $V-S$ uma CV

Considere $u, v \in S$

$(u, v) \notin E$

portanto S é CI

Exercício

Seja $G = (V, E)$ um grafo não-direcionado
mostre que CI e conjunto independente são equivalentes

Seja U um conjunto de elementos, e S é um conjunto de subconjuntos de U . Um **set cover** é um subconjunto de S cuja união é U .

→ mostra que há uma redução entre cobertura de vértices e set cover

→ estes problemas são equivalentes? não são equivalentes, quando fazemos o inverso

$$U = \{1, 2, 3, 4, 5\}$$

$$S = \begin{cases} A = \{1, 2\} \\ B = \{3, 4\} \\ C = \{2, 4\} \end{cases} \quad \text{set cover} = \begin{cases} A, B, C \\ \text{ou} \\ A, B \end{cases}$$

Resolução:

$$\text{Seja } S = \{S_i \subseteq U\}$$

Vértices $\subseteq$ Conjunto

$$E \rightarrow U$$

$$r \rightarrow S$$

$$\forall e = (u, r) \in E \implies e \in S_u \text{ e } e \in S_r$$

$|C|$

1 - não

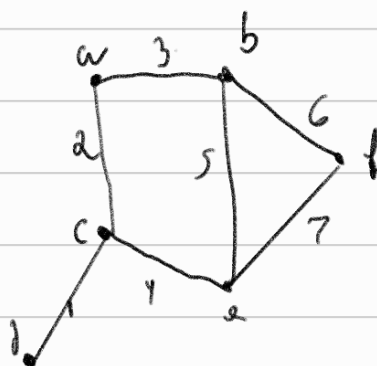
2 - não

3 - Sim

4 - Sim

5 - Sim

6 - Sim



Abertura de vértices

↳ eliminação

↳ tem tempo a ser exponencial

$O(2^n)$

eficiência está diretamente ligada a custo polinomial

Seja A um subconjunto de V e k um inteiro positivo

A é uma coleção de vértices de cardinalidade $\leq k$?

Polinomial

Não podemos afirmar isso apesar de alguma coisa verdadeira

2ª pergunta

Seja A um subconjunto de V e k um inteiro positivo
existe uma coleção de vértices de cardinalidade $\leq k$?

Polinomial

Problema! não dá pra relacionar A

3ª pergunta

Seja $G=(V,E)$ e $k \geq 0$

existe uma coleção de vértices de cardinalidade $\sum_{x \in A} k$?

$\leq c$

Polinomial

NP ^{problema}
problema

que alguém consegue verificar se uma determinada resposta é verdadeira ou falsa

NP (não determinístico polinomial) $\rightarrow$ Verifica em tempo polinomial

$\underbrace{\text{Verificabilidade}}_{\text{é possível reduzir}}$
 $\text{Verificabilidade} \subseteq \text{Conjunto}$
 $\underbrace{\text{Conjunto}}_{\text{não é possível reduzir}}$

$\Downarrow$ Conclusão

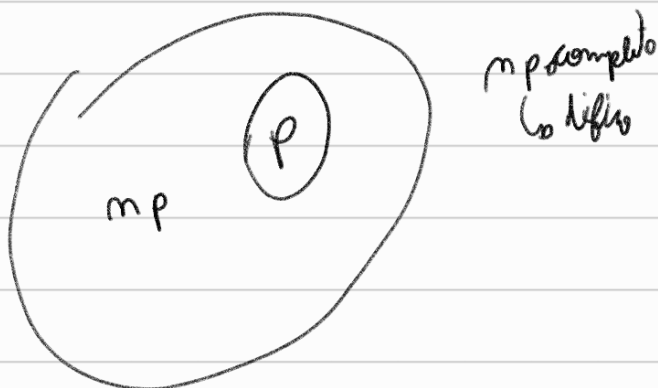
Teorema Kurti

Transformar a instância de um problema Transformar em outra
 e com o oráculo não obter a resposta original tudo polinomial
 Pode chamar muitas vezes

Transformações de Cook

verificar problemas entre métodos e falso e verdadeiro
 em uma única chamada

coot é mais genérico do que coon



Seja $S = \{(p_i, p_j)\}$ um conjunto de intervalos. Dois intervalos $A_k = (p_i^k, p_j^k)$ e $A_l = (p_i^l, p_j^l)$ são disjuntos

Se a interseção entre eles é vazia

Projetar ^{pelo eixo x} soluções para "maximizar" um subconjunto de S em que seus intervalos sejam mutuamente disjuntos

$(2, 5) \quad (3, 7) \quad (6, 8) \quad (9, 12) \quad (1, 2)$
 $S^1 = (2, 5) \quad (6, 8) \quad (9, 12)$
 $S^2 = (1, 2) \quad (6, 8) \quad (9, 12)$
 $S^3 = \quad \quad \quad 1, 3 \quad 2, 4 \quad 3, 5$

- 1 - Escolhe o que começa primeiro
- 2 - Escolher por menor duração do intervalo
- 3 - Escolher os que tem menor numero de conflitos
- 4 - Escolhe por seleção mesmo sem ordenar nem nada
- 5 - Seleção aleatória

Sua solução é ótima?

↳ mostre ou dê um contra-exemplo que ilustra a não otimalidade

Solução aleatória → pode ser que o intervalo escolhido tenha um número muito grande de conflitos o que inviabiliza a possibilidade de maximizar o número de intervalos

Escolher o que começa primeiro → pode ser que o intervalo comece antes dos outros mas tenha uma duração muito longa resultando em conflitos com outros intervalos

Escolher o que tem menor numero de conflitos → talvez a melhor solução não seja a final

Escolher os de menor duração → pode ser que os intervalos de menor

dentado tenham conflito entre si, ou que tenham conflito com
outros intervalos maiores

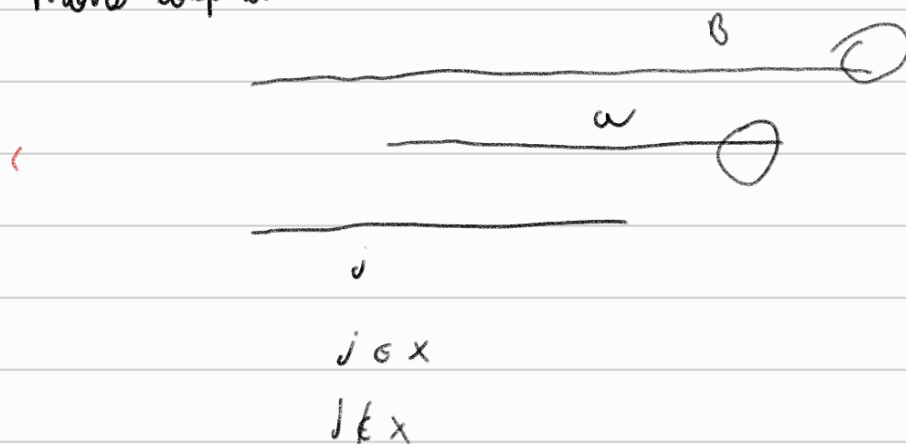
Solução ótima o tempo de término: ordenar pelo tempo de término
e ir escolhendo os que não tenham conflitos

↓ f_{im} ↑ t_{termina}

quando termina primeiro

junta tempo de início e o término

Prova completa:



Algoritmo guloso nunca $\rightarrow$ tem um foco
não voltam atrás nas tomadas de decisão

Exercício 5 ILV10

① Seja F uma fila de um banco

Considere que cada cliente i possui um tempo de atendimento t_i

Considere que n clientes chegam ao mesmo tempo. Como organizar a fila para
minimizar o tempo médio que os n clientes fiquem no banco?

Justifique

organizar a fila de forma crescente. Uma vez que tempo de espera
de um cliente é o tempo que ele vai gastar esperando os outros na sua frente

② Seja uma prova de PPA

Considere que não é possível fazer a prova todo no tempo especificado

Como escolher as questões de forma a maximizar sua nota

Fazer considerações que julga necessárias

Selecionar questões baseir e como menor tempo de duração.

③ Fazer um algoritmo "Guloso" que pode ser aplicado para qualquer problema que pode ser resolvido usando abordagem gulosa

Para minimizar, sempre escolher o item com menor tempo de execução primeiro
Maximizar, ao contrário

Inicialização

While

escolher ótimo local
adicione
remova

Minimização local x global

$S \leftarrow \emptyset$

While Há elemento para analisar ou não chegou

$x \leftarrow \text{escolha_melhor}()$

if (x for viável)

$S \leftarrow S \cup \{x\}$

-10

else

remova x (e autor que tem problema com x)

atualização em remove

① Sejam n arquivos ordenados

Como "fundir" estes arquivos de forma que o arquivo resultante esteja ordenado

Faça as considerações que julgar necessárias

② Seja n intervalos. Cada intervalo possui tempo de início, fim e ganho

Projete uma solução para escolher k intervalos disjuntos maximizando o ganho

1- MergeSort: Intercalar os arquivos, escolhendo sempre o menor valor entre os n arquivos | Divisão e conquista

Minimiza o custo computacional

2- Greedy ordenar pelo tempo de fim

1 10 1
1-3 5-7 9-15

③ Conjunto de elementos que está presente em todos os vetores

• Dado n vetores, encontre o conjunto de elementos que está presente em todos os vetores

$u_1: 3, 10, 15, 30, 100,$
 $u_2: 30, 45, 68, 70$
 $u_3: 30, 45, 100, 150$

n vetores

Selecionar os 2 de menor tamanho
achar interseção e fazer isso até ter
um único vetor resultante

$1, 3, 10, 15, 30, 30, 30$
 $\left\{ \begin{array}{l} 1, 20, 45 \\ 1, 68, 100 \\ 1, 70, 150 \end{array} \right.$

$n * k$

30,

$3, 20, 45$
 $3, 68, 100,$
 $3, 70, 150$

$10, 20, 45$
 $10, 45, 100$
 $10, 68, 150$
 $10, 70$

Exercício

Seja $M = \{1, 4, 6\}$ um conjunto de moedas em que cada quantidade é ilimitada.
Seja $x=8$ o valor a ser atingido.

Projetar uma solução ótima e eficiente para encontrar e usar a quantidade de moedas para atingir x

Dividir o valor a ser atingido, por cada quantidade de moeda partindo da moeda de maior valor e usar o resto para continuar dividindo por outros valores

Algoritmo guloso

$$\left\{ \begin{array}{l} \text{ex: } 8 \div 6 = 2 \quad 8 \overline{) 6} \quad 2 \overline{) 4} \quad 2 \overline{) 1} \\ \quad 2 \div 4 = 2 \quad \quad \quad 2 \overline{) 1} \\ \quad 2 \div 1 = 0 \end{array} \right.$$

Porém essa não é a solução ótima, que seriam 2 moedas e no resultado acima o resultado são 3 moedas

$$V[0] = 0$$

$$V[1] = V[0] + i = 1$$

$$V[2] = V[1] + i = 2$$

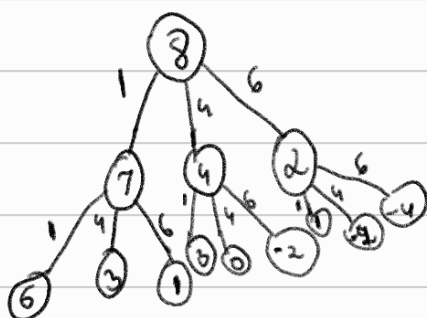
$$V[3] = V[2] + i = 3$$

$$V[4] = V[3] + i = 4$$

X

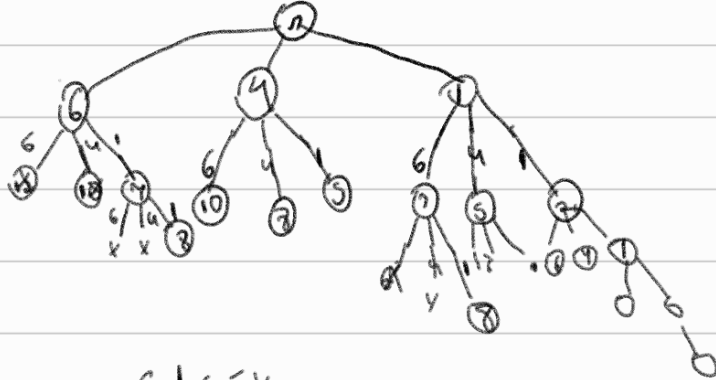
Parte 2: Retornando a parte de ótima e eficiente como resolver com uma árvore

Podem ser
ESTRATÉGIA Top Down



Backtracking

branching - nodes



Precisa de mais árvore?

$$6 + 6 = X$$

$$6 + 4 = X$$

$$6 + 1 = 7$$

$$7 +$$

$$\sqrt{10} =$$

$$6 +$$

$$6 + 4 = X$$

$$6 + 1 = 7$$

$$7 + 1 = 8$$

$$4 +$$

$$4 + 6 = X$$

$$4 + 4 = 8$$

$$4 + 1 = 5$$

$$5 + 6 = X$$

$$5 + 4 = X$$

$$5 + 1 = 6$$

$$1 +$$

$$1 + 6 = 7$$

$$1 + 4 = 5$$

$$1 + 1 = 2$$

Sejam M um conjunto de moedas e S um valor a ser atingido
 Como selecionar um subconjunto de moedas para atingir S ?
 → MINIMIZE A QUANTIDADE

```
int resolve (...){
```

```
    if (s == 0) return 0;
```

```
    else if (s < 0) return 0;
```

```
    resposta = +∞
```

```
    for (int i = 0; i < m; i++) {
```

```
        x = resolve (... , s - moeda[i])
```

```
        if (x != -1)
```

```
            resposta = min(x + 1, resposta)
```

```
}
```

Seja S um conjunto de intervalos e que cada intervalo possui um ganho associado

Projetar uma solução para encontrar um subconjunto de S , de forma a maximizar o ganho, sendo os intervalos mutuamente disjuntos

Forçar uma solução ótima e eficiente

Ordenar todos intervalos pelo tempo de término

Qual o maior ganho possível ter até o intervalo $i-1$

opção 1: $\text{max_ganhos} = \text{arr}[i-1]$

último intervalo

ganho y

opção 2: Se pegarmos o y não pode pegar nenhum intervalo que se sobreponha a ele

Encontrar o último intervalo que termina antes de y

o ganho será o ganho de y + o melhor lucro possível já calculado

$$\text{ganhoComJ} = \text{ganho}[y] + \text{arr}[j]$$

Após o cálculo dos 2 cenários, selecionar o cenário de maior ganho

$$\text{max_Gain} = \max(\text{GanhosenJ}, \text{GanhoComJ})$$